

# AlexNet实现鸟类识别

## 一 实验内容

本实验将采用 AlexNet 实现鸟类图片的识别分类

## 二 实验目标

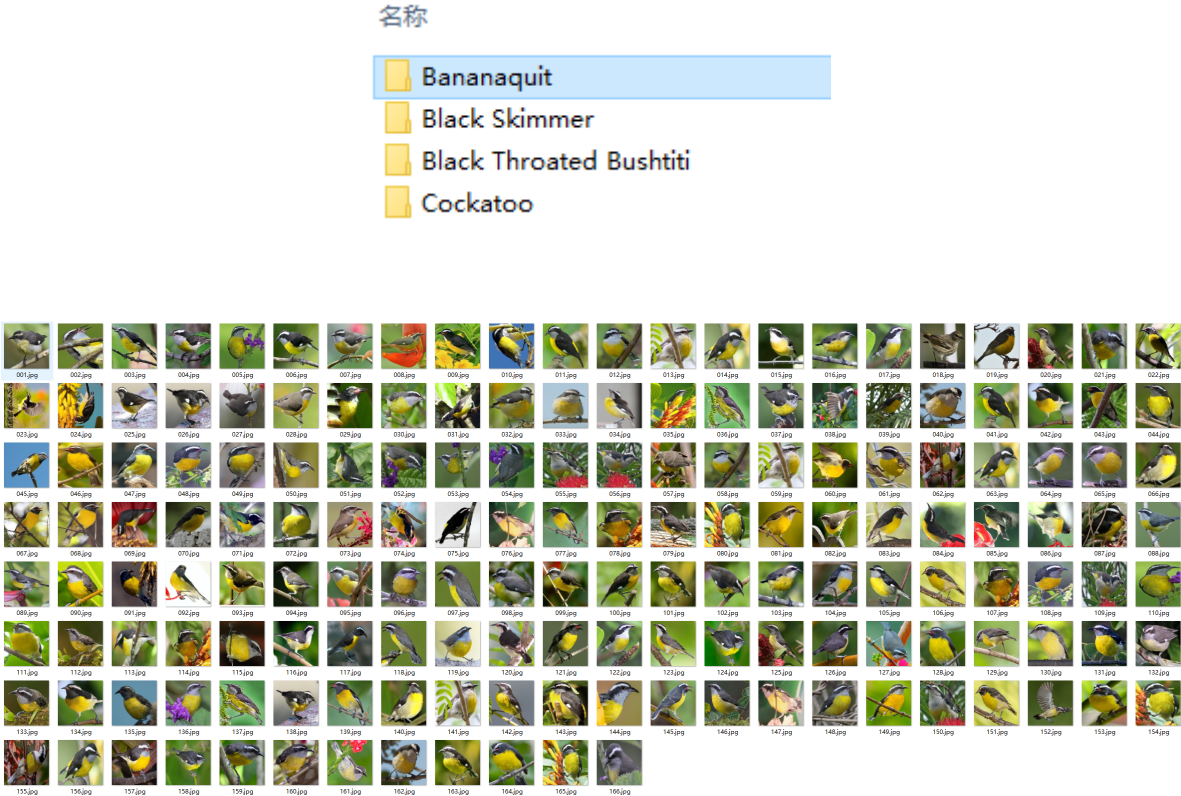
- 1. 了解鸟类图像数据集
- 2. 掌握卷积神经网络模型搭建、训练、预测、模型评估的完整流程 **重点**
- 3. 熟练使用tensorflow.keras构建卷积神经网络 **重点**
- 4. 熟练利用卷积神经网络解决图像分类任务 **难点**

## 三 实验环境

- 操作系统：ubuntu18 及以上
- 语言环境：Python3.6.5
- 编译器：jupyter notebook
- 深度学习环境：TensorFlow2.4.1
- 显卡（GPU）：NVIDIA GeForce RTX 3090

## 四 实验原理

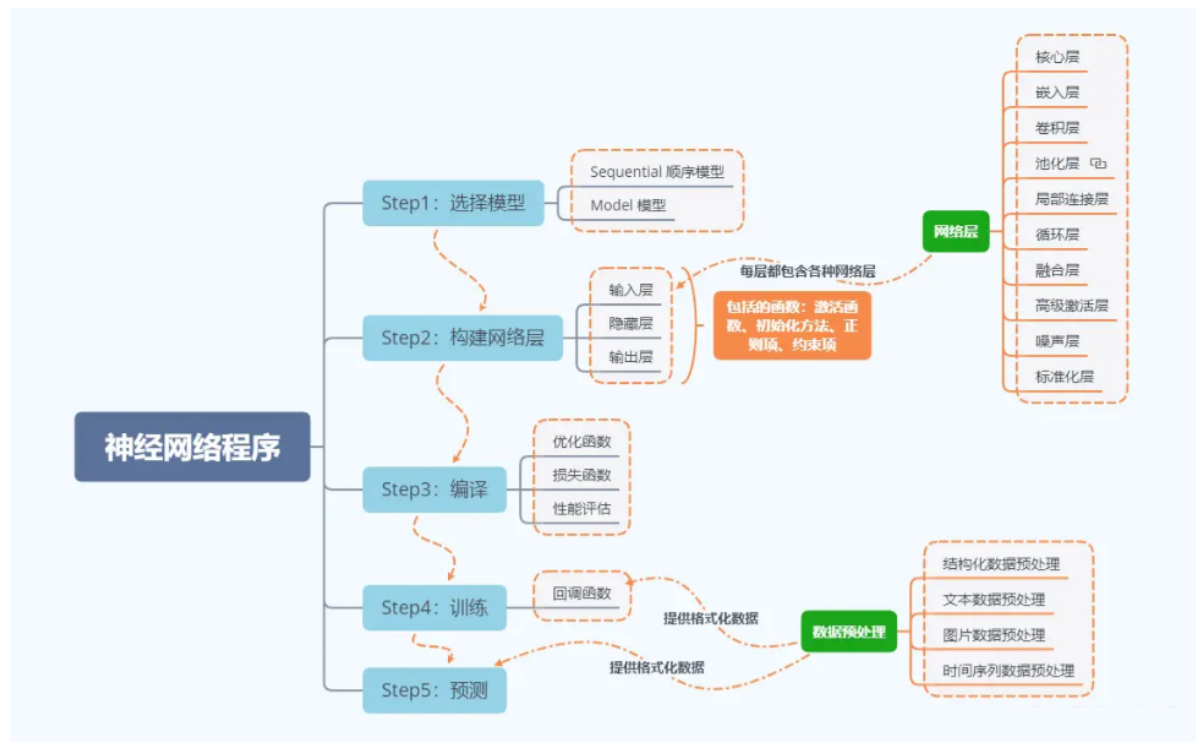
### 1. 鸟类图像数据集介绍



- 共包含4类鸟的图像，共565张图像

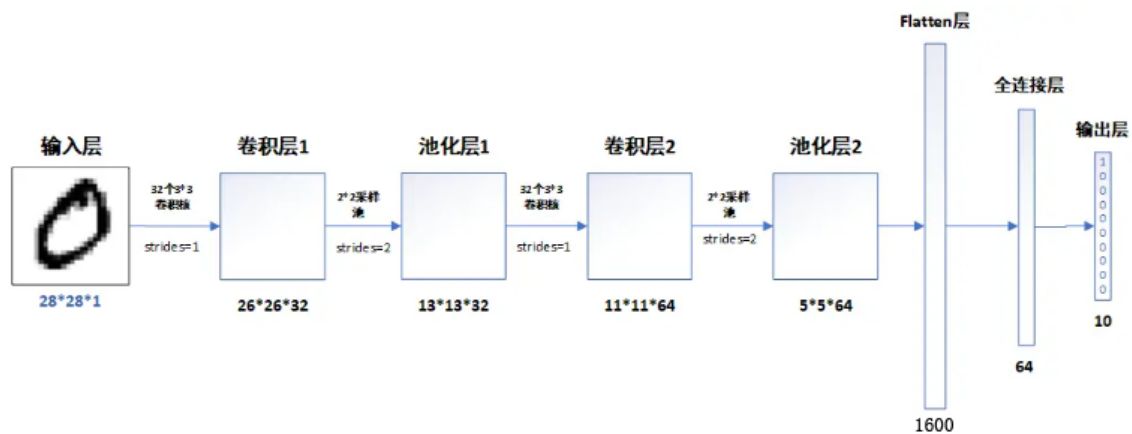
- 图像为彩色三通道,图像宽高: 224x224

## 2. 神经网络程序说明



## 3. 网络结构说明

### 模型的结构



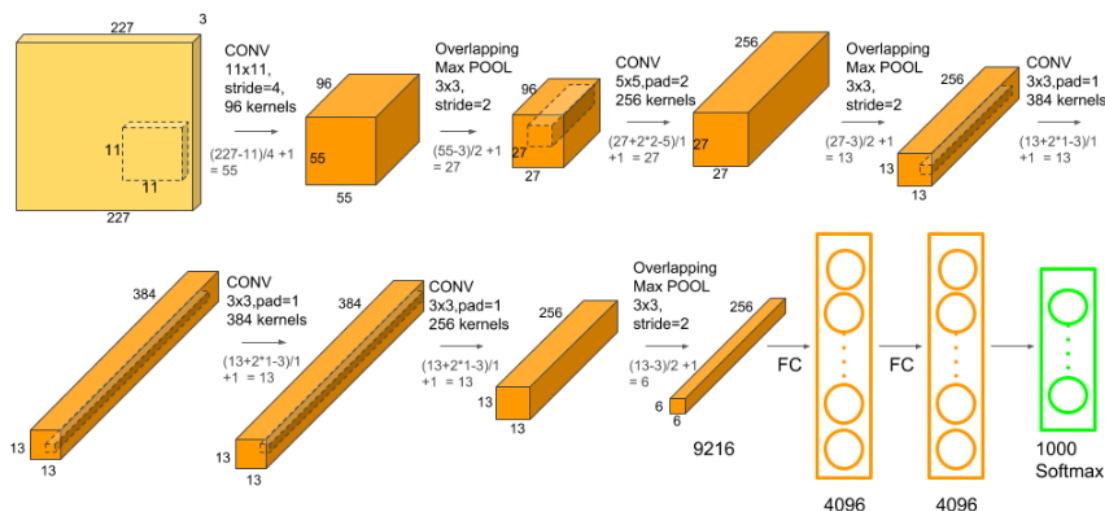
### 各层的作用

- 输入层: 用于将数据输入到训练网络
- 卷积层: 使用卷积核提取图片特征
- 池化层: 进行下采样, 用更高层的抽象表示图像特征
- Flatten层: 将多维的输入一维化, 常用在卷积层到全连接层的过渡
- 全连接层: 起到“特征提取器”的作用
- 输出层: 输出结果

## 4. AlexNet (8层) 介绍

AlexNet使用了ReLU方法加快训练速度，并且使用Dropout来防止过拟合

AlexNet (8层) 是首次把卷积神经网络引入计算机视觉领域并取得突破性成绩的模型。获得了ILSVRC 2012年的冠军，再top-5项目中错误率仅仅15.3%，相对于使用传统方法的亚军26.2%的成绩优良重大突破。和之前的LeNet相比，AlexNet通过堆叠卷积层使得模型更深更宽。



## 五 实验步骤

### 一、前期准备

设置GPU (如果使用的是CPU可以忽略这步)

```
import tensorflow as tf

gpus = tf.config.list_physical_devices("GPU")

if gpus:
    tf.config.experimental.set_memory_growth(gpus[0], True) #设置GPU显存用量按需使用
    tf.config.set_visible_devices([gpus[0]], "GPU")
```

导入数据

```
import matplotlib.pyplot as plt
import pathlib

# 支持中文
plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号

import os, PIL
```

```
# 设置随机种子尽可能使结果可以重现
import numpy as np
np.random.seed(1)

# 设置随机种子尽可能使结果可以重现
import tensorflow as tf
tf.random.set_seed(1)

data_dir = "../dataset/bird_photos"

data_dir = pathlib.Path(data_dir)
```

## 查看数据

```
image_count = len(list(data_dir.glob('*/*')))

print("图片总数为: ",image_count)
```

输出结果

图片总数为: 565

## 二、数据预处理

| 文件夹                     | 数量    |
|-------------------------|-------|
| Bananaquit              | 166 张 |
| Black Throated Bushtiti | 111 张 |
| Black skimmer           | 122 张 |
| Cockatoo                | 166张  |

### 1. 加载数据

使用 `image_dataset_from_directory` 方法将磁盘中的数据加载到 `tf.data.Dataset` 中

```
batch_size = 8
img_height = 227
img_width = 227
```

TensorFlow版本是2.2.0的同学可能会遇到 `module 'tensorflow.keras.preprocessing' has no attribute 'image_dataset_from_directory'` 的报错，升级一下TensorFlow就OK了。

```
"""
关于image_dataset_from_directory() 生成训练集数据
"""
train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size=(img_height, img_width),
    batch_size=batch_size)
```

输出结果:

```
Found 565 files belonging to 4 classes.
Using 452 files for training.
```

```
"""
关于image_dataset_from_directory() 生成验证集数据
"""
val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=(img_height, img_width),
    batch_size=batch_size)
```

输出结果

```
Found 565 files belonging to 4 classes.
Using 113 files for validation.
```

我们可以通过class\_names输出数据集的标签。标签将按字母顺序对应于目录名称。

```
class_names = train_ds.class_names
print(class_names)
```

输出结果:

```
['Bananaquit', 'Black Skimmer', 'Black Throated Bushtiti', 'Cockatoo']
```

## 2. 可视化数据

```
plt.figure(figsize=(10, 5)) # 图形的宽为10高为5
plt.suptitle("Bird Identification")

for images, labels in train_ds.take(1):
    for i in range(8):

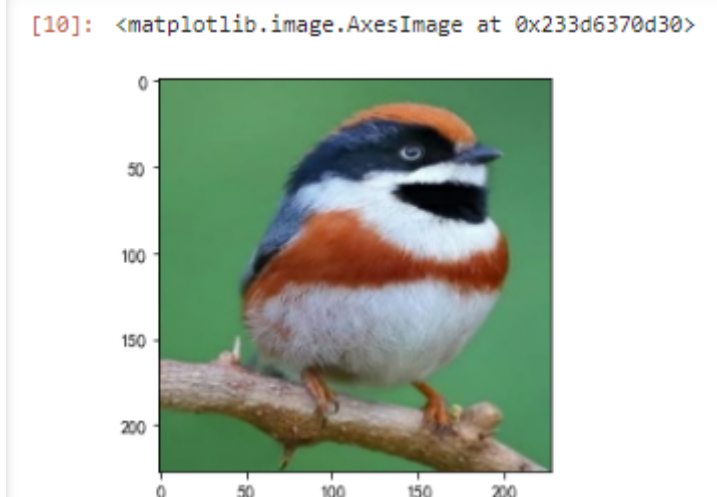
        ax = plt.subplot(2, 4, i + 1)

        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])

        plt.axis("off")
```



```
plt.imshow(images[1].numpy().astype("uint8"))
```



### 3. 再次检查数据

```
for image_batch, labels_batch in train_ds:
    print(image_batch.shape)
    print(labels_batch.shape)
    break
```

输出结果

```
(8, 227, 227, 3)
(8,)
```

- `Image_batch` 是形状的张量 (8, 224, 224, 3)。这是一批形状240x240x3的8张图片（最后一维指的是彩色通道RGB）。
- `Label_batch` 是形状 (8, ) 的张量，这些标签对应8张图片

#### 4. 配置数据集

- `shuffle()`：打乱数据
- `prefetch()`：预取数据，加速运行，其详细介绍可以参考我前两篇文章，里面都有讲解。
- `cache()`：将数据集缓存到内存当中，加速运行

```
AUTOTUNE = tf.data.AUTOTUNE

train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.cache().prefetch(buffer_size=AUTOTUNE)
```

### 三、构建AlexNet (8层) 网络模型

下面是实验的重点，可以试着按照上面三张图自己构建一下ResNet-50

```
from tensorflow.keras import layers, models, Input
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dense, Flatten,
Dropout, BatchNormalization, Activation

import numpy as np
seed = 7
np.random.seed(seed)

def AlexNet(nb_classes, input_shape):
    input_tensor = Input(shape=input_shape)

    # 1st block
    x = Conv2D(96, (11,11), strides=4, name='block1_conv1')(input_tensor)
    x = BatchNormalization()(x)
    x = Activation('relu')(x)
    x = MaxPooling2D((3,3), strides=2, name = 'block1_pool')(x)

    # 2nd block
    x = Conv2D(256, (5,5), padding='same', name='block2_conv1')(x)
    x = BatchNormalization()(x)
    x = Activation('relu')(x)
    x = MaxPooling2D((3,3), strides=2, name='block2_pool')(x)

    # 3rd block
    x = Conv2D(384, (3,3), activation='relu',
padding='same', name='block3_conv1')(x)
    # 4th block
    x = Conv2D(384, (3,3), activation='relu',
padding='same', name='block4_conv1')(x)

    # 5th block
```

```

x = Conv2D(256, (3,3), activation='relu',
padding='same',name='block5_conv1')(x)
x = MaxPooling2D((3,3), strides=2, name = 'block5_pool')(x)

# full connection
x = Flatten()(x)
x = Dense(4096, activation='relu', name='fc1')(x)
x = Dropout(0.5)(x)
x = Dense(4096, activation='relu', name='fc2')(x)
x = Dropout(0.5)(x)
output_tensor = Dense(nb_classes, activation='softmax', name='predictions')
(x)

model = Model(input_tensor, output_tensor)
return model

model=AlexNet(1000, (img_width, img_height, 3))
model.summary()

```

输出结果:

Model: "model"

| Layer (type)                 | Output Shape          | Param #  |
|------------------------------|-----------------------|----------|
| =====                        |                       |          |
| input_1 (InputLayer)         | [(None, 227, 227, 3)] | 0        |
| block1_conv1 (Conv2D)        | (None, 55, 55, 96)    | 34944    |
| batch_normalization (BatchNo | (None, 55, 55, 96)    | 384      |
| activation (Activation)      | (None, 55, 55, 96)    | 0        |
| block1_pool (MaxPooling2D)   | (None, 27, 27, 96)    | 0        |
| block2_conv1 (Conv2D)        | (None, 27, 27, 256)   | 614656   |
| batch_normalization_1 (Batch | (None, 27, 27, 256)   | 1024     |
| activation_1 (Activation)    | (None, 27, 27, 256)   | 0        |
| block2_pool (MaxPooling2D)   | (None, 13, 13, 256)   | 0        |
| block3_conv1 (Conv2D)        | (None, 13, 13, 384)   | 885120   |
| block4_conv1 (Conv2D)        | (None, 13, 13, 384)   | 1327488  |
| block5_conv1 (Conv2D)        | (None, 13, 13, 256)   | 884992   |
| block5_pool (MaxPooling2D)   | (None, 6, 6, 256)     | 0        |
| flatten (Flatten)            | (None, 9216)          | 0        |
| fc1 (Dense)                  | (None, 4096)          | 37752832 |
| dropout (Dropout)            | (None, 4096)          | 0        |

|                     |              |          |
|---------------------|--------------|----------|
| fc2 (Dense)         | (None, 4096) | 16781312 |
| <hr/>               |              |          |
| dropout_1 (Dropout) | (None, 4096) | 0        |
| <hr/>               |              |          |
| predictions (Dense) | (None, 1000) | 4097000  |
| <hr/>               |              |          |

Total params: 62,379,752  
 Trainable params: 62,379,048  
 Non-trainable params: 704

## 四、训练模型

```

epochs = 20

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=epochs
)

```

输出结果:

```

Epoch 1/20
57/57 [=====] - 5s 30ms/step - loss: 9.2789 - accuracy:
0.2166 - val_loss: 3.2340 - val_accuracy: 0.3363
Epoch 2/20
57/57 [=====] - 1s 14ms/step - loss: 0.9329 - accuracy:
0.6224 - val_loss: 1.1778 - val_accuracy: 0.5310
Epoch 3/20
57/57 [=====] - 1s 14ms/step - loss: 0.7438 - accuracy:
0.6747 - val_loss: 1.9651 - val_accuracy: 0.5133
Epoch 4/20
57/57 [=====] - 1s 14ms/step - loss: 0.8875 - accuracy:
0.7025 - val_loss: 1.5589 - val_accuracy: 0.4602
Epoch 5/20
57/57 [=====] - 1s 14ms/step - loss: 0.6116 - accuracy:
0.7424 - val_loss: 0.9914 - val_accuracy: 0.4956
Epoch 6/20
57/57 [=====] - 1s 15ms/step - loss: 0.6258 - accuracy:
0.7520 - val_loss: 1.1103 - val_accuracy: 0.5221
Epoch 7/20
57/57 [=====] - 1s 13ms/step - loss: 0.5138 - accuracy:
0.8034 - val_loss: 0.7832 - val_accuracy: 0.6726
Epoch 8/20
57/57 [=====] - 1s 14ms/step - loss: 0.5343 - accuracy:
0.7940 - val_loss: 6.1064 - val_accuracy: 0.4602
Epoch 9/20
57/57 [=====] - 1s 14ms/step - loss: 0.8667 - accuracy:
0.7606 - val_loss: 0.6869 - val_accuracy: 0.7965
Epoch 10/20
57/57 [=====] - 1s 16ms/step - loss: 0.5785 - accuracy:
0.8141 - val_loss: 1.3631 - val_accuracy: 0.5310
Epoch 11/20

```

```

57/57 [=====] - 1s 15ms/step - loss: 0.4929 - accuracy:
0.8109 - val_loss: 0.7191 - val_accuracy: 0.7345
Epoch 12/20
57/57 [=====] - 1s 15ms/step - loss: 0.4141 - accuracy:
0.8507 - val_loss: 0.4962 - val_accuracy: 0.8496
Epoch 13/20
57/57 [=====] - 1s 15ms/step - loss: 0.2591 - accuracy:
0.9148 - val_loss: 0.8015 - val_accuracy: 0.8053
Epoch 14/20
57/57 [=====] - 1s 15ms/step - loss: 0.2683 - accuracy:
0.9079 - val_loss: 0.5451 - val_accuracy: 0.8142
Epoch 15/20
57/57 [=====] - 1s 14ms/step - loss: 0.2925 - accuracy:
0.9096 - val_loss: 0.6668 - val_accuracy: 0.8584
Epoch 16/20
57/57 [=====] - 1s 14ms/step - loss: 0.4009 - accuracy:
0.8804 - val_loss: 1.1609 - val_accuracy: 0.6372
Epoch 17/20
57/57 [=====] - 1s 14ms/step - loss: 0.4375 - accuracy:
0.8446 - val_loss: 0.9854 - val_accuracy: 0.7965
Epoch 18/20
57/57 [=====] - 1s 14ms/step - loss: 0.3085 - accuracy:
0.8926 - val_loss: 0.6477 - val_accuracy: 0.8761
Epoch 19/20
57/57 [=====] - 1s 15ms/step - loss: 0.1200 - accuracy:
0.9538 - val_loss: 1.8996 - val_accuracy: 0.5398
Epoch 20/20
57/57 [=====] - 1s 15ms/step - loss: 0.3378 - accuracy:
0.9095 - val_loss: 0.9337 - val_accuracy: 0.8053

```

## 五、模型评估

```

acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

loss = history.history['loss']
val_loss = history.history['val_loss']

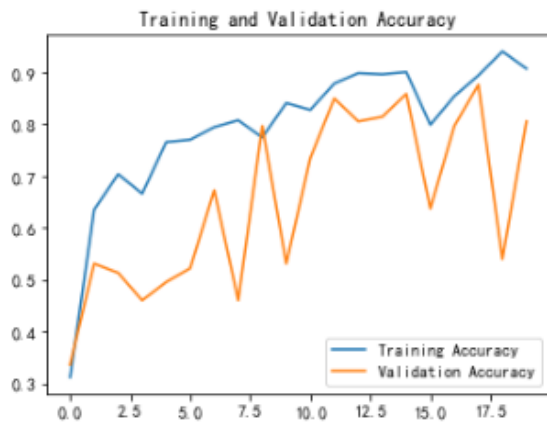
epochs_range = range(epochs)

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.suptitle("Bird Identification")

plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()

```



## 七、保存&加载模型

这是最简单的模型保存与加载方法哈

```
# 保存模型
model.save('model/my_model.h5')
# 加载模型
new_model = tf.keras.models.load_model('model/my_model.h5')
```

## 八、预测

```
# 采用加载的模型（new_model）来看预测结果

plt.figure(figsize=(10, 5)) # 图形的宽为10高为5
plt.suptitle("Bird identification")

for images, labels in val_ds.take(1):
    for i in range(8):
        ax = plt.subplot(2, 4, i + 1)

        # 显示图片
        plt.imshow(images[i].numpy().astype("uint8"))

        # 需要给图片增加一个维度
        img_array = tf.expand_dims(images[i], 0)

        # 使用模型预测图片中的人物
        predictions = new_model.predict(img_array)
        plt.title(class_names[np.argmax(predictions)])

        plt.axis("off")
```



## 六 实验总结

- 本实验利用CNN构建了8层网络 (AlexNet)

CNN灵活应用与解决图像识别问题，完整使用流程如下：

